

Video title: Training, Validation, and Test Split in PyTorch (Module 4)

Code snippets reference

This document lists the code snippets shown in the video along with brief descriptions. We recommend reviewing this document alongside the video for the best learning experience.

Disclaimer:

The code snippets shown in this video are for conceptual illustration only. They are simplified or partial examples designed to explain the underlying ideas and may not run as-is. Please avoid copying the code directly without understanding the context and structure required to make it work.

S. No.	Code	Description
1.	<pre>import torch import torch.nn as nn import torch.optim as optim model = nn.Linear(1,1) criterion = nn.MSELoss() optimizer = optim.SGD(model.parameters(), lr=0.01) for epoch in range(100): y_hat = model(x_train) loss = criterion(y_hat, y_train) optimizer.zero_grad() loss.backward() optimizer.step()</pre>	Slide title: Training dataset Purpose: Runs forward pass, computes loss, backpropagates gradients, and updates model parameters.
2.	<pre>with torch.no_grad(): y_val_pred = model(x_val)</pre>	Slide title: Validation dataset Purpose: Performs forward pass on validation data and

	<pre> val_loss = criterion(y_val_pred, y_val) print("Validation Loss:", val_loss.item()) </pre>	computes validation loss without updating parameters.
3.	<pre> with torch.no_grad(): y_test_pred = model(x_test) test_loss = criterion(y_test_pred, y_test) print("Test Loss:", test_loss.item()) </pre>	Slide title: Test dataset Purpose: Performs forward pass on test data and computes test loss without updating parameters.
4.	<pre> torch.save(model.state_dict(),"linear_model.pth") </pre>	Slide title: Saving and loading model parameters Purpose: Stores the model's learned parameters to a file.
5.	<pre> model.load_state_dict(torch.load("linear_model.pth")) model.eval() </pre>	Slide title: Saving and loading model parameters Purpose: Loads saved parameters into the model and sets it to evaluation mode.